Automate the Boring Stuff with Python - 3rd Edition by Al Sweigart

Chapters 1-8 - Python Review

- Chapters 1-8 were a fine Python review, but the Python Tutorial is readily available, so I didn't find these chapters noteworthy. I did like the example of the logging library, a clever way to store a chessboard in a dictionary, and the pig latin translator. So I have reproduced those here for review. They are all in the ch01-08 directory, but I put a logging demo here since it's both the shortest and the most useful.

```
#!/usr/bin/env python
import logging

logging.basicConfig(level=logging.DEBUG, format=' %(asctime)s - %(levelname)s -
%(message)s')
# You could also specific a file with filename='myProgramLog.log' within
basicConfig

logging.debug("some minor code and debugging details.")
logging.info("An even happened")
logging.warning("Something could go wrong.")
logging.error('An error has occurred.')
logging.critical("The program is unable to recover!")

print ("Now again, but only error and above is displayed!")
logging.disable(logging.WARNING)
logging.debug("some minor code and debugging details.")
logging.info("An even happened")
logging.warning("Something could go wrong.")
logging.error('An error has occurred.')
logging.critical("The program is unable to recover!")
```

- As the book points out, using `logging` is superior to debugging with print. For one, when you are done with print debugging you have to go back and remove the print statements for debugging while keeping those for necessary program output. With `logging` messages, you just disable logging (or store it to a log) and leave program output alone.
- Remember, `logging` messages are intended more for the programmer than the user. For error messages that the user should see, use a `print()` call. Also, you can disable all

logging with `logging.disable(logging.CRITICAL)`, and you can do it right after the `basicConfig` line to be program wide.

Chapter 9 - Text Pattern Match with Regular Expressions

- Define your regex (with Humre or raw strings), then pass it to re.compile, and search or findall with the result. Search returns a match object or None and the match object has a group method. With no number of 0, group returns the entire match. findall returns a list of tuples of the groups of the match if there were groups or a list of matches otherwise. Instead of search/findall, you can call sub('REPLACEMENT_TEXT', 'Text to be substituted into.').
- humre is normally imported with `from humre import *`
- `humre.parse(r'regex')` will give you humre for the regex.

```
import re
from humre import *
phone_regex_nonverbose = r'((\d{3}|\(\d{3}\))?(\s|-|\.)?(\d{3})(\s|-|\.)(\d{4})
(\s*(ext|x|ext\.)\s*(\d{2,5}))?)'
phone_regex = re.compile(r'''(
    (\d{3}|\(\d{3}\))?  # Area code
    (\s|-|\.)?  # Separator
    (\d{3})  # First three digits
    (\s|-|\.)  # Separator
    (\d{4})  # Last four digits
    (\s*(ext|x|ext\.)\s*(\d{2,5}))?  # Extension
    )''', re.VERBOSE)
phone_humre = group(
    optional_group(either(exactly(3, DIGIT),  # Area code
                          OPEN_PAREN + exactly(3, DIGIT) + CLOSE_PAREN)),
    optional(group_either(WHITESPACE, '-', PERIOD)),  # Separator
    group(exactly(3, DIGIT)),  # First three digits
    group_either(WHITESPACE, '-', PERIOD),  # Separator
    group(exactly(4, DIGIT)),  # Last four digits
    optional_group(  # Extension
      zero_or_more(WHITESPACE),
      group_either('ext', 'x', r'ext\.'),
      zero_or_more(WHITESPACE),
      group(between(2, 5, DIGIT))
      )
)
```

- Regex Review and Humre Introduction
- Humre Package Page

Chapter 10 - Reading and Writing Files

- See an os specific path with `str(Path('Users', 'Lewis', 'bin'))` after using `from pathlib import Path`. We normally always use forward slashes and convert with `Path` and many file functions take a Path without needing it to become a `str`.
- As long as one of the first two values in an expression are a Path object, you can use `/` to join paths or string folder names to paths.
- `Path.cwd()` returns current working directory and `os.chdir()` changes directory.
- `Path.home()` returns the home directory.
- Use `os.makedirs('/Users/Lewis/git/books/python/new_dir')` to make a directory and all intermediate directories.
- Instead us the `mkdir` method on a `Path` object to make a directory from a `Path` object. It only makes one directory unless you pass it `parents=True`.
- The `is_absolute` method of a `Path` object tells you if a `Path` is absolute and the `absolute` method returns an absolute version of the `Path`.
- A `Path` has various attributes representing parts of the filepath including `anchor`, `parent`, `name`, `stem`, and `suffix` plus `drive` on Windows. These are all strings except `parent` which is a path.
- The `stat` method of a `Path` will give you file statistics such as `st_size` and `st_mtime`.
- `Path` objects of folders have a `glob` method for listing content matching a pattern.
- `Path` objects also have `exists`, `is_file`, and `is_dir` methods
- `Path` objects have a `read_text` method that returns the entire file contents as a string and a `write_text` method that writes or overwrites a file with a string.
- You can get a `File` object with `read`, `write`, and `readLines` methods by using the built in `open` command. After reading or writing a file, use the `close` method on it. The second positional argument to `open` is `r`, `w`, or `a` for mode. A useful `open` keyword argument is `encoding` which should usually be `'UTF-8'` which is the default on non-Windows OSs.
- `open` accepts a string path or a `Path` object.
- you can use `with open` to automatically close your files outside a block:

```
with open('data.txt', 'w', encoding='UTF-8') as file_obj:
    file_obj.write("Hello, world!")
with open('data.txt', encoding='UTF-8') as file_obj:
    content = file_obj.read()
```

- a `shelve` file stores variables as if it was a dictionary.

```
import shelve
shelf_file = shelve.open('mydata')
shelf_file['cats'] = ['Zophie', 'Pooka', 'Simon']
shelf_file.close()
```

Chapter 11 - Organizing Files

- The `shutil` (shell utilities) module has functions to copy, move, rename, and delete files.
- `shutil.copy(source, destination)` copies a file to another folder or another filename and `shutil.copytree(source, destination)` copies a folder with all its files and subfolders to another folder, and `shutil.move(source, destination)` will move a file or folder.
- There are several ways to delete. `shutil.rmtree(path)` removes a folder and all it's files or subfolders, `os.unlink(path)` will delete a single file at path, and `os.rmdir(path)` will delete an empty folder at path.
- The third party `sendtotrash` module will delete files and folders to the Recycle Bin.
- files and subfolders of a folder can be listed with `os.listdir(path)` or the `iterdir` method of a `Path` object.
- `os.walk(path)` will walk a directory tree. It returns a string of the current folders name, a list of strings of the subfolders in the current folder, and a list of string of the files in the current folder on every iteration.
- The `zipfile` module includes the `ZipFile` object and handy methods to work with zip files.

```
import zipfile
with open('file1.txt', 'w', encoding='utf-8') as file_obj:
    file_obj.write('Hello' * 10000)
with zipfile.ZipFile('example.zip', 'w') as example_zip:
    example_zip.write('file1.txt', compress_type=zipfile.ZIP_DEFLATED,
                      compresslevel=9)
```

- The `ZipFile` also has `namelist` to list contents and `extractall` to extract contents and can give you a `ZipInfo` though `getinfo(filename)`. `extractall(path)` extracts to path where without path is uses current working directory. There is also an `extract(filename)` and `extract(filename, path)` to extract individual files. Destination folders will be created if they do not already exist.

Chapter 12 - Designing and Deploying Command Line Programs

- Run `python -m venv .venv` to create a virtual environment and `source .venv/bin/activate` to activate it or `.venv\Scripts\activate.bat` on Windows. Run `.venv/bin/deactivate` or `.venv\Scripts/deactivate.bat` to disable it or close the Terminal.
- `python -m pip list` will list installed Python packages with version.
- `python -m pip install package_name` will install packages in the active environment.
- To upgrade to the latest version run `python -m pip install -U package_name` or to install a particular version use `python -m pip install package_name==1.17.4` for version 1.17.4.
- Uninstall a package with `python -m pip uninstall package_name` and access help with `python -m pip --help`.
- There are several useful variables that give your program information about itself.
    - `__file__` contains the .py file's path as a string and `Path(__file__)` will give you an object referring to your script.
    - `sys.executable` contains the full path to the Python interpreter.
    - `sys.version` contains the Python interpreter version and `sys.version_info.major` / `sys.version_info.minor` contain the major and minor version numbers of the Python interpreter (ex: For Python 3.13.1, major is 3 and minor is 13). Also `list(sys.version_info)` returns `[3, 13, 1, 'final', 0]`.
    - `os.name` contains `nt` on Windows as `posix` on Mac/Linux.

    - `sys.platform` is more specific and contains `win32`, `darwin`, or `linux`.
    - The platform module contains highly specific OS version and CPU type information.
    - You can check for a module by putting your import in a try catch block:

```
try:
    import nonexistentModule
except ModuleNotFoundError:
    print('This code runs if nonexistentModule was not found.')
```

    - If a module is necessary, you can put a descriptive error message in the `except` block and call `sys.exit()` to terminate the program.

- If you call `python3 yourScript.py hello world` then `sys.argv` contains `['yourScript.py', 'hello world']`.
- If you are going to handle a lot of command line arguments, use `argparse` documentation. It's equivalent to `Flag` in Go. For something more like Cobra, use either Click which is decorator based or Typer which uses type hints.
- There isn't a universal Python equivalent to Viper, but Dynaconf handles multilayered configuration and python-dotenv is good for simpler cases.
- `pyperclip.copy()` puts a string on the clipboard and `pyperclip.paste()` fetches the clipboard contents as a string. It needs to be installed via `pip` and requires `xclip` on Linux.
- You can produce colorful terminal output with `Bext` (installed via pip). It only works in terminal windows and gives you `bext.fg('red')` and `bext.bg('green')` commands to set foreground and background colors of the next print. `bext.bg('reset')` makes text normal again. It contains other features: `bext.clear()` to clear the screen, `bext.width()` and `bext.height()` to return current width (in columns) and height (in rows) of the terminal window, `bext.hide()` and `bext.show()` to show and hide the cursor, `bext.title(text)` to set the title bar to a text string, `bext.goto(x, y)` to move the cursor where 0, 0 is the top-left, and `bext.get_key()` which waits for any key and returns the key pressed like a single-key version of `input()`.
- You can clear the screen without Bext with this one-liner:

```
import os
def clear():
    os.system('cls' if os.name == 'nt' else 'clear')
```

- `playsound3.playsound('hello.mp3')` will play an audiofile if you install `playsound3` via pip.
- `pymsgbox` (installed via pip) lets you easily add small GUI message boxes to your program using Tkinter. `pymsgbox.alert(text)` which returns 'OK', `pymsgbox.confirm(text)` which returns 'OK' or 'Cancel', `pymsgbox.prompt(text)` which returns typed text or None if they clicked Cancel, and `pymsgbox.password(text)` which is the same as `prompt` but masks the input on the screen; these are all provided by `pymsgbox`.
- You can easily make batch files to run your scripts:
  - On Windows:

```
@call %HOMEDRIVE%%HOMEPATH%\Scripts\.venv\Scripts\activate.bat
@python %HOMEDRIVE%%HOMEPATH%\Scripts\yourScript.py %*
@pause
@deactivate
```

  - On Mac:

```
source ~/Scripts/.venv/bin/activate
python3 ~/Scripts/yourScript.py
deactivate
```

  - On Linux:

```
#!/usr/bin/env bash
source ~/Scripts/.venv/bin/activate
python3 ~/Scripts/yourScript.py
read -p "Press any key to continue..." -n1 -s
deactivate
```

  - Note: On Mac and Linux you need to run `chmod u+x scriptName` to make the script executable.
  - A more elegant solution than sourcing the venv in a script is to use the following shebang then make the .py file executable.

```
#!/path/to/your/project/.venv/bin/python
```

- `pyinstaller` (installed via pip) lets you turn a .py script into a standalone executable with `python -m PyInstaller --onefile yourScript.py`, but you must run PyInteraller on the OS you want the executable to run on. PyInstaller Documentation.

Chapter 13 - Web Scraping

- The `webbrowser` module in the standard library does one thing, `webbrowser.open('https://inventwithpython.com/')` will open the site in a new tab.
- The `requests` module (installed with pip) lets you download files from the web
- `requests.get(url)` returns a `response` object with fields such as `response.status_code` which you can compare to `requests.codes.ok` and `response.text` which contains the entire response body.
- Rather than checking the response code, you can call `response.raise_for_status()` to raise an `HTTPError` exception if the connection failed. It does nothing if the download was successful.

```
import requests
response = requests.get('https://inventwithpython.com/page_that_does_not_exist')
# It is normally fine to let the Exception crash the program, but here we are
# catching it and displaying a nicer error message
try:
    response.raise_for_status()
except Exception as exc:
    print(f'There was a problem: {exc}')
```

- When saving a web page to a file, you should `open` it in `wb` (write binary) mode to preserve encoding. You can also use the `Response` object's `iter_content()` method to work a line at a time.

```
import requests
response = requests.get('https://automatetheboringstuff.com/files/rj.txt')
response.raise_for_status()
with open('RomeoAndJuliet.txt', 'wb') as play_file:
    for chunk in response.iter_content(100000):
        play_file.write(chunk)
```

- See more about `requests` including the `response.json()` method at Requests Documentation.
- Don't use regex's to parse html; use an HTML modules such as `bs4`.
- You can right click or Ctrl+click to Inspect Element to see a webpage element source, then right click on the element in the developer console and select `Copy -> CSS Selector` to get the CSS Selector to pass to Beautiful Soup's `select()` method or Selenium's `find_element()` method.
- Use `beautifulsoup4` to install Beautiful Soup via `pip` but `import bs4`.
- The `bs4.BeautifulSoup()` function takes a string containing HTML and returns a `BeautifulSoup` object.
- You can use `requests` to fetch a `response.text` or open a file on your hard drive.
- `bs4.select()` takes a CSS selector and returns a list of `Tag` object which represent HTML elements.

```
import bs4
example_file = open('example3.html')
example_soup = bs4.BeautifulSoup(example_file.read(), 'html.parser')
elems = example_soup.select('#author')  # elems is a list of Tag objects
print(elems[0].get_text())              # elems[0] is a Tag with get_text method
```

- `Tag` elements have `get_text` to get the text value and `attrs` with all HTML attributes as a dictionary. They can be passed to `str` to show the HTML tags they represent. The `get` method allows you to pass an attribute to access values.

```
# searchpypi.py - Opens several search results on pypi.org

import requests, sys, webbrowser, bs4

print('Searching...')   # Display text while downloading the search results
page.
res = requests.get('https://pypi.org/search/?1=' + ' '.join(sys.argv[1:]))
res.raise_for_status()

# Retrieve top search result links.
soup = bs4.BeautifulSoup(res.text, "html.parser")
# Open a browser tab for each result.
link_elems = soup.select('.package-snippet')
num_open = min(5, len(link_elems))
for i in range(num_open):
    url_to_open = 'https://pypi.org' + link_elems[i].get('href')
    print('Opening', url_to_open)
    webbrowser.open(url_to_open)
```

Selenium

- `requests` and `BeautifulSoup` are powerful, but sometimes you need a traditional browser where scripts can't go, and that's what Selenium is for.
- To import Selenium, use `from selenium import webdriver` then launch firefox with `browser = webdriver.Firefox()` and go to a page with `browser.get(url)`. It also supports `browser.back()`, `browser.forward()`, `browser.refresh()`, and `browser.quit()`.
- `browser.find_element(query)` returns the first match for your query and `browser.find_elements(query)` returns every matching element.
- query can use a number of varieties, but first you must `from selenium.webdriver.common.by import By` to get the `By` object.
  - `By.CLASS_NAME` - Elements that use the CSS class name.
  - `By.CSS_SELECTOR` - Elements that use the CSS selector.
  - `By.ID` - Elements with a matching id attribute value.
  - `By.LINK_TEXT` - <a> elements that completely match the text provided.
  - `By.PARTIAL_LINK_TEXT` - <a> elements that contain the text provided.
  - `By.NAME` - Elements with a matching name attribute value.
  - `By.TAG_NAME` - Elements with a matching tag name (case-insensitive).

- A query that matches no elements throws a `NoSuchElement` exception.
- The find methods return `WebElement` or lists of `WebElement` with the following methods and attributes:
  - `tag_name` - The tag name, such as 'a' for an <a> element.
  - `click()` - simulates a mouse click on the element.
  - `send_keys()` - used to fill in textbox and textareas, or do things like send `Keys.HOME` or `Keys.END` to scroll the browser to the top of bottom of the page.
  - `get_attribute(name)` - The value for an element's name attribute, like href in an <a> element.
  - `get_property(name)` - The value for the element's property, which does not appear in the HTML code. Examples include `innerHTML` and `innerText`.
  - `text` - The text within the element, such as `'hello'` in `<span>hello</span>`.
  - `clear()` - For text field or text area elements, clears the text entered into it.
  - `is_displayed()` - Returns `True` if the element is visible.
  - `is_enabled()` - For input elements, returns `True` if the element is enabled.
  - `is_selected()` - For checkbox or radio button elements, returns `True` if the element is selected.
  - `location` - A dictionary with 'x' and 'y' keys for the position of the element on the page.
  - `size` - A dictionary with 'width' and 'height' keys for the size of the element on the page.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
browser = webdriver.Firefox()
browser.get('https://author.com/example3.html')
elems = browser.find_elements(By.CSS_SELECTOR, 'p')
print(elems[0].text)
print(elems[0].get_property('innerHTML'))
link_elem = browser.find_element(By.LINK_TEXT, 'This text is a link')
# We're going to stay on the page and not follow the link
# link_elem.click() # Follows the "This text is a link" link
user_elem = browser.find_element(By.ID, 'login_user')
user_elem.send_keys('your_username_here')
password_elem = browser.find_element(By.ID, 'login_pass')
password_elem.send_keys('your_password_here')
password_elem.submit()
```

- If you `from selenium.webdriver.common.keys import Keys` you can use a variety of key presses with `send_keys`:
  - `Keys.ENTER, Keys.PAGE_UP, Keys.DOWN, Keys.RETURN, Keys.ESCAPE, Keys.LEFT`
  - `Keys.HOME, Keys.BACK_SPACE, Keys.RIGHT, Keys.END, Keys.DELETE, Keys.TAB`
  - `Keys.PAGE_DOWN, Keys.UP, Keys.F1 to Keys.F12`

Playwright

- Playwright is newer than Selenium and runs in headless mode.
- Playwright installs webdrivers through `python -m playwright install`.

```
from playwright.sync_api import sync_playwright
with sync_playwright() as playwright:
    browser = playwright.firefox.launch()
    page = browser.new_page()
    page.goto('https://author.com/example3.html')
    print(page.title())
    browser.close()
```

- Use `playwright.chromium.launch()` or `playwright.webkit.launch()` to use Chrome or Safari instead.
- Playwright is harder to debug since it uses headless mode and works in a with block, but you can interact with it at a REPL like:

```
from playwright.sync_api import sync_playwright
playwright = sync_playwright().start()
browser = playwright.firefox.launch(headless=False, slow_mo=50)
page = browser.new_page()
page.goto('https://author.com/example3.html')
browser.close()
playwright.stop()
```

- The `Page` object returned by `browser.new_page()` has a number of methods for basic browser functions in addition to `goto` such as `page.go_back()`, `page.go_forward()`, `page.reload()`, and `page.close()`.
- The `Page` object also has ways to find elements on the page called *locators* which return `Locator` objects. If the element you specified doesn't exist, Playwright pauses for 30 seconds while it waits for the element to appear. To force it to check immediately instead of

timing out, you can call `is_visible()` on the `Locator`. You can also call `page.query_selector('selector')` where *selector* is a string of the element's CSS or XPath selector and it immediately returns, returning `None` if it found no matches.
    - `page.get_by_role(role,name=label)` - elements by their role and optionally their *label*.
    - `page.get_by_text(text)` - elements that contain *text* as part of their inner text.
    - `page.get_by_label(label)` - elements with matching <label> text as *label*.
    - `page.get_by_placeholder(text)` - input and textarea elements with matching `placeholder` attribute values as *text*.
    - `page.get_by_alt_text(text)` - img elements with matching `alt` attribute values as *text*.
    - `page.locator(selector)` - elements with a matching CSS or XPath *selector*.
- `Locator` objects have a variety of methods.
    - `get_attribute(name)` - returns the value of an element's *name* attribute, such as 'https://nostarch.com' for the `href` attribute in an a element.
    - `count()` - returns an integer of the number of matching elements in this `Locator`.
    - `nth(index)` - returns a `Locator` of the matching element by index. For example `nth(3)` returns the fourth matching element.
    - `first` - returns the `Locator` of the first matching element.
    - `last` - returns the `Locator` of the last matching element.
    - `all()` - returns a list of `Locator` objects for each matching element.
    - `inner_text()` - returns the text within the element, such as `'hello'` in `<b>hello</b>`.
    - `inner_html()` - returns the html source within the element, such as `'<b>hello</b>'` in `<b>hello</b>`.
    - `click()` - simulates a click on the element.
    - `is_visible()` - returns `True` if the element is visible.
    - `is_enabled()` - for input elements, returns `True` if the element is enabled.
    - `is_checked()` - For checkbox or radio button elements, returns `True` if the element is selected.
    - `bounding_box()` - returns a dictionary with keys 'x' and 'y' for position of the element's top-left corner in the page, along with 'width' and 'height' for the element's size.
- A simple example:

```
from playwright.sync_api import sync_playwright
with sync_playwright() as playwright:
    browser = playwright.firefox.launch(headless=False, slow_mo=50)
    page = browser.new_page()
    page.goto('https://author.com/example3.html')
    elems = page.locator('p')
    print(elems.nth(0).inner_text())
    print(elems.nth(0).inner_html())
```

- Clicking Elements on the Page

```
from playwright.sync_api import sync_playwright
playwright = sync_playwright().start()
browser = playwright.firefox.launch(headless=False, slow_mo=50)
page = browser.new_page()
page.goto('https://author.com/example3.html')
page.click('input[type=checkbox]')  # Checks the checkbox
page.click('input[type=checkbox]')  # Unchecks the checkbox
page.click('a')     # Clicks the link
page.go_back()
checkbox_elem = page.get_by_role('checkbox')
checkbox_elem.check()       # Checks the checkbox
checkbox_elem.uncheck()     # Unchecks the checkbox
checkbox_elem.set_checked(True)       # Checks the checkbox
checkbox_elem.set_checked(False)      # Unchecks the checkbox
page.get_by_text('is a link').click() # Uses a locator method
browser.close()
playwright.stop()
```

- Filling Out and Submitting Forms

```
from playwright.sync_api import sync_playwright
playwright = sync_playwright().start()
browser = playwright.firefox.launch(headless=False, slow_mo=50)
page = browser.new_page()
page.goto('https://author.com/example3.html')
page.locator('#login_user').fill('your_username_here')
page.locator('#login_pass').fill('your_password_here')
page.locator('input[type=submit]').click()
browser.close()
playwright.stop()
```

- Sending Special Keys

```
from playwright.sync_api import sync_playwright
playwright = sync_playwright().start()
browser = playwright.firefox.launch(headless=False, slow_mo=50)
page = browser.new_page()
page.goto('https://author.com/example3.html')
page.locator('html').press('End')   # Scrolls to bottom
page.locator('html').press('Home')  # Scrolls to top
browser.close()
playwright.stop()
```

- You can pass a number of strings to `press()`
    - 'Backquote', 'Escape', 'ArrowDown'
    - 'Minus', 'End', 'ArrowRight'
    - 'Equal', 'Enter', 'ArrowUp'
    - 'Backslash', 'Home', 'F1' to 'F12'
    - 'Backspace', 'Insert', 'Digit0' to 'Digit9'
    - 'Tab', 'PageUp', 'KeyA' to 'KeyZ'
    - 'Delete', 'PageDown'
- For more about Playwright, see Playwright's Documentation.

Chapter 14 - Excel Spreadsheets

- The `openpyxl` module (available from pip) lets you read and modify Excel spreadsheet files. openpyxl Documentation.
- Open a file with `wb = openpyxl.load_workbook('example3.xlsx')`, get a lists of sheets with `wb.sheetnames`, access a sheet with `sheet = wb['Sheet1']` or `sheet = wb.active` for the active sheet, then access cells with `sheet['A1'].value`. You can find the row, column, and value of a cell with `c = sheet['B1']`: `print(f'Row {c.row}, Column {c.column} is {c.value}`. There is also a `c.coordinate` that tells you it is cell 'B1'. You can also get a cell with `c = sheet.cell(row=1, column=2)` instead of 'B1' which works well with `range`. Note that `cell` takes the integer 2 for column and not the letter 'B'.
- A worksheet object has a `max_row` and `max_column` attribute. Both are numeric.
- You can use `openpyxl.utils.get_column_letter(num)` to convert numbers to letters and `openpyxl.utils.column_index_from_string('letter')` for vice versa:

- You can takes slices of sheets like `sheet['A1':'C3']` and interate over them:

```
import openpyxl
wb = openpyxl.load_workbook('example3.xlsx')
for row_of_cell_objects in sheet['A1':'C3']:
    for cell_obj in row_of_cell_objects:
        print(cell_obj.coordinate, cell_obj.value)
    print('--- END OF ROW ---')
```

- The above will print a row at a time: A1-C1, then A2-C2, then A3-C3
- You can also loop over all the cell objects in a column with `for cell_obj in list(sheet.columns)[1]:`. A sheet also has `sheet.rows` which is list-like and can be forced into a `list`. The `rows` and `columns` attribute is only indexable when passed to list(), and they both give us a list of tuples containing `Cell` objects. `list(sheet.rows)` would give us all seven rows of three columns each for our example spreadsheet.

```
#!/usr/bin/env python
# readCensusExcel.py - Tabulates county population and census tracks

import pprint

import openpyxl

print("Opening workbook...")
wb = openpyxl.load_workbook("censuspopdata.xlsx")
sheet = wb["Population by Census Tract"]
# This is to be a dictionary of state abbreviation keys mapping to
# county name which in turn are a dictionary mapping to 'tracts' and 'pop'
# Ex: {'AK': {'Aleutians East': {'pop': 3141, 'tracts': 1},
#             {'Aleutians West': {'pop': 5561, 'tracts': 2}, ...
county_data = {}

print("Reading rows...")
for row in range(2, sheet.max_row + 1):
    # Each row in the spreadsheet has data for one census tract.
    state = sheet["B" + str(row)].value
    county = sheet["C" + str(row)].value
    pop = sheet["D" + str(row)].value

    # Make sure the key for this state exists.
    county_data.setdefault(state, {})
    # Make sure the key for this count in this state exists.
    county_data[state].setdefault(county, {"tracts": 0, "pop": 0})

    # Each row represents one census tract, so increment by one.
    county_data[state][county]["tracts"] += 1
    # Increase the county pop by the pop in this census tract.
    county_data[state][county]["pop"] += int(pop)

# Open a new text file and write the contents of county_data to it.
# We are writing this to a Python file we could later `import census2010`
# and access `census2010.allData['AK']['Aleutians East']`
print("Writing results...")
result_file = open("census2010.py", "w")
result_file.write("allData = " + pprint.pformat(county_data))
result_file.close()
print("Done.")
```

- `openpyxl` also provides ways of writing data. You make changes to your sheet then call `wb.save('filename.xlsx')` to save changes.

- `wb.create_sheet()` will make a new sheet, and `wb.create_sheet(index=0, title='First Sheet')` will make a new 'First Sheet' at index 0. You can delete a sheet with `del wb['Sheet1']`. Editing a cell value is as easy as `sheet['A1'] = 'Hello, world!'`

```
#!/usr/bin/env python
# updateProduce.py - Corrects costs in produce sales spreadsheet

import openpyxl

wb = openpyxl.load_workbook("produceSales3.xlsx")
sheet = wb["Sheet"]

# The produce types and their updated prices
PRICE_UPDATES = {"Garlic": 3.07, "Celery": 1.19, "Lemon": 1.27}

# Loop through the rows and update the prices.
for row_num in range(2, sheet.max_row + 1):  # Skip the first row.
    produce_name = sheet.cell(row=row_num, column=1).value
    if produce_name in PRICE_UPDATES:
        sheet.cell(row=row_num, column=2).value = PRICE_UPDATES[produce_name]

wb.save("updatedProduceSales3.xlsx")
```

- To customize font styles in cells, `from openpyxl.styles import Font`, then you can set `sheet['A1'].font = Font(size=24, italic=True)`. Font takes `name` (string), `size` (int), `bold`, and `italic` (both boolean). You can assign your `Font` object to a variable and then assign the variable to any cell `font` attribute.
- You can assign formulas to cells like `sheet['B9'] = '=SUM(B1:B8)'`
- A worksheet has `row_dimensions` and `col_dimensions` that control row heights and column widths. You access values by numerically indexing into the `row_dimensions` and using a column letter to index into `col_dimensions`.
- A worksheet has `merge_cells` and `unmerge_cells` methods that take a range of cells
- A worksheet also has a `freeze_panes` attribute that you can set to a `Cell` object or a string of a cell's coordinates to freeze all rows above this cell and all columns to the left of it, but not the row and column of the cell itself. `sheet.freeze_panes = 'A2'` freezes Row 1 and no columns. Very useful with headers. 'B1' would freeze column A and no rows. Frozen columns and rows always are visible as you scroll through the data.
- `openpyxl` also supports creating `BarChart`, `LineChart`, `ScatterChart`, and `PieChart` objects. The book gives some quick details. It's a five step process.

Chapter 15 - Google Sheets

- To use Google Sheets you need a Google Cloud project with Google Sheets and Google Drive APIs with OAuth configured. The book goes in to detail for this one time process.
- If I ever want to work with Google Sheets, I should re-read this chapter on `ezsheets`
- EzSheets Documentation

Chapter 16 - Sqlite Databases

- SQLite is a full SQL relational database that can handle even TBs of data and multiple simultaneous read operations. Its primary drawback is that it can't efficiently handle hundreds or thousands of simultaneous write operations (for example, from a social media app). It also lacks permission settings and user roles; no GRANT or REVOKE statements. While columns do have data types, it's also of note that it doesn't strictly enforce column data types.
- You can name a sqlite file anything, but by convention we user the .db ending. The extension .sqlite is also common. The entire database lives in one file.
- The first step is to connect: `conn = sqlite3.connect('example.db', isolation_level=None)`. When you call `connect` it will create the file given if it does not exist or throw a `sqlite3.DatabaseError` if the file exists but is not a database. `isolation_level=None` is a second parameter to `connect` that tells it to autocommit. When you are finished with the `Connection` call `conn.close()`.
- It is an unenforced convention that SQL keywords be all uppercases and identifiers such as table and column names be lowercase and separate multiple words with underscores.
- If you try to run `CREATE TABLE` without `IF NOT EXISTS` and the table exists, the statement raises an `sqlite3.OperationalError`.
- There are five SQLite datatypes: `NULL`, `INT` or `INTEGER`, `REAL`, `TEXT`, and `BLOB`. The datatypes aren't strictly enforced, but SQLite will try to cast data to the columns datatype if possible. However, if you create the table with the `STRICT` keyword at the end, SQLite will raise `sqlite3.IntegrityError` if you try to insert data of the wrong type within the table. A '42' in an 'INT' will be inserted as numeric 42, but a 'Hello' will raise an exception. `STRICT` was added in SQLite 3.37.0 which is used by Python 3.11 and later.
- Notably missing are Boolean and date/time/datetime datatypes. Use an `INTEGER` for Boolean and store 1 or 0 and use a TEXT field for dates and store like `YYYY-MM-DD`, `YYYY-MM-DD HH:MM:SS`, `YYYY-MM-DD HH:MM:SS.SSS`, `HH:MM:SS`, or `HH:MM:SS.SSS`.
- SQLite tables automatically create a `rowid` column contains a unique primary key integer.

- The `sqlite_schema` table includes metadata about the database. You can get a list of tables with `conn.execute('SELECT name FROM sqlite_schema WHERE type="table"').fetchall()`. To obtain information about the columns in the 'cats' table run `conn.execute('PRAGMA TABLE_INFO(cats)').fetchall()`. The column information tuple lists column position (1 would be the second column in the table), name, datatype, whether or not the column is NOT NULL (0 is False, 1 is True), default value, and whether the column is the primary key (0 is False, 1 is True). These `fetchall()` examples assume we are at a REPL and just want to view a Tuple of results. In code you would store it or loop over the resulting rows.
- The `sqlite3.sqlite_version` string contains your SQLite version.
- You can use `fetchall()` from the returned `Cursor` to fetch all the results of a query, or you can loop over rows returns in a `for` statement

```
import sqlite3
conn = sqlite3.connect('example.db', isolation_level=None)
conn.execute('CREATE TABLE IF NOT EXISTS cats (name TEXT NOT NULL, birthdate
TEXT, fur TEXT, weight_kg REAL) STRICT')
conn.execute('INSERT INTO cats VALUES ("Zophie", "2021-01-24", "black", 5.6)')
conn.execute('INSERT INTO cats VALUES ("Toby", "2021-05-07", "black", 6.8)')
conn.execute('UPDATE cats SET fur = "gray tabby" WHERE name="Zophie"')
for row in conn.execute('SELECT * FROM cats'):
    # The row is a tuple of column values from the query.
    print('Row data:', row)
    print(row[0], "is one of my favorite cats")
```

- In addition to `NOT NULL`, SQLite supports `UNIQUE` constraints on columns and any `UNIQUE` column can participate in a foreign key relation. You can also declare `UNIQUE` on a combination of fields such as putting `UNIQUE (name, birthdate)` at the end of the table definition after the last column and that combination of fields can participate in a foreign key.
- `SELECT *` returns all columns except `rowid`. To get `rowid` you must ask for it. Remember, `SELECT` returns a list of tuples or an empty list.
- You can limit `SELECT` results with a `WHERE` clause. SQLite uses `=, !=, <, >, <=, >=, AND, OR and NOT`. Note it uses `=` for equality and not `==`.
- SQLite also suport `ORDER BY` in `ASC` or `DESC` order after the `WHERE` clause and (for example) `LIMIT 3` after the `ORDER BY` to limit to 3 results.
- Note that SQLite allows single or double quotes around strings; usually whatever you aren't using to delimit the Python string the query is being provided in.

- `conn.execute('CREATE INDEX idx_cats_name ON cats (name)')` will create a `name` index. Indexes are often worthwhile for frequent queries. You pay for faster lookups with slower inserts and updates though. Index names are global for the database, which is why we named it `idx_table_column` instead of just `idx_column`. You can drop an index by executing `DROP INDEX idx_name`. Starting index names with `idx_` is a common convention. You don't often need indexes before you hit 1000 or more rows.
- To see all indexs for a table use `conn.execute('SELECT name FROM sqlite_schema WHERE type = "index" AND tbl_name = "cats"').fetchall()`
- To avoid SQL injection, parameterize queries with `?` instead of concatenated strings or using Python format strings.
- Always include a `WHERE` clause in your `UPDATE` queries to help avoid global updates. If you intend to do a global update, use `WHERE 1`.
- In autocommit mode, a new transaction normally starts and commits everytime you run `execute()`, but you can instead run `conn.execute('BEGIN')` to start a new transaction and it will continue until you call `conn.commit()` or `conn.rollback()`.
- To backup a database not in use, just copy the file or even programmatically `shutil.copy('example.db', 'backup.db')`. But if your database is always in use, you can connect to another database file (Ex: `backup_conn = sqlite3.connect('backup.db', isolation_level=None)`) and use `conn.backup(backup_conn)`
- SQLite supports all the standard SQL operations I haven't mentioned: `DELETE`, `ALTER`, `DROP`; all through `conn.execute`. `ALTER` however is somewhat limited, and it is common to create/copy/drop/rename:

```
CREATE TABLE new_table (-- new schema here);
INSERT INTO new_table SELECT * FROM old_table;
DROP TABLE old_table;
ALTER TABLE new_table RENAME TO old_table;
```

- To use foreign keys, you need to `conn.execute('PRAGMA foreign_keys = ON')` then you could `conn.execute('CREATE TABLE IF NOT EXISTS vaccinations (vaccine TEXT, date_administered TEXT, administered_by TEXT, cat_id INTEGER, FOREIGN KEY(cat_id) REFERENCES cats(rowid)) STRICT')`. The `PRAGMA` enforces not being able to insert for a non-existent foreign key value and not being able to delete a row in the foreign table while rows exists in the keyed table (you have to delete `vaccinations` before deleting a `cat`. You need to run the `PRAGMA` after calling `sqlite3.connect()` to get these safeties enforced for the session.

- It is a good practice to always run `conn.execute('PRAGMA foreign_keys = ON')` after you call `sqlite3.connect()` and you could easily combine the two in a helper function.
- You can create an in memory database with `memory_db_conn = sqlite3.connect(':memory:', isolation_level=None)` but you will still need a `file_db_conn` pointing at a file so you can save your in memory database with `memory_db_conn.backup(file_db_conn)`. It can be an amazing performance boost. However, if your program crashes without calling `backup()` you lose data, so you need to be careful to backup the db in your exception handling.
- You can call `iterdump()` on your `Connection` object to generate the text of the SQLite queries needed to re-create the database.

```
import sqlite3
conn = sqlite3.connect('sweigartcats.db', isolation_level=None)
with open('sweigartcats-queries.txt', 'w', encoding='utf-8') as fileObj:
    for line in conn.iterdump():
        fileObj.write(line + '\n')
```

- The `sqlite3` command is documented at https://sqlite.org/cli.html and is useful for interacting with your database without having to write a program. `sqlite3` is preinstalled on Mac OS X.
- You can also use other database tools to interact with a SQLite database including DB Browser, SQLite Studio, DBeaver Community, and even Datagrip.
- One special feature of `sqlite3` worth mentioning is the row_factory:

```
conn.row_factory = sqlite3.Row
```

- A `row_factory` allows you to access tuple values by column name instead of positionally, like `row['name']` instead of `row[0]`. It makes code much more readable.
- For more information see The SQLite Documentation and the sqlite3 library documentation.

Chapter 17 - PDF and Word Documents

PDF

- `pypdf` (available via pip) is a Python package for creating and modifying PDF files. PDF's are complicated, so even `pypdf` may make mistakes when exacting text or fail to open

particular pdf files. `pdfminer` is an older library that may have more success with difficult files.

```
#!/usr/bin/env python

import pypdf
import pdfminer.high_level

PDF_FILENAME = "Recursion_Chapter1.pdf"
TEXT_FILENAME = "recursion.txt"
text = ""

try:
    reader = pypdf.PdfReader(PDF_FILENAME)
    for page in reader.pages:
        text += page.extract_text()
except Exception:
    text = pdfminer.high_level.extract_text(PDF_FILENAME)
with open(TEXT_FILENAME, "w", encoding="utf-8") as file_obj:
    file_obj.write(text)
```

- You can clean up text by feeding it through an LLM with a prompt such as: "The following is text extracted from several pages of a PDF of a book on recursive algorithms. Clean up this text. By this, I mean put paragraphs on a single, separate line. Also remove the footer and header text from each page. Also get rid of the hyphens at the end of each line for words split up across the line. Do not make any spelling, grammar corrections, or rewording. Here is the text:"
- If you don't want to use an LLM, there are pypdf Post-processing Tips.
- In addition to `PdfReader`, `pypdf` has `PdfWriter`. However, `PdfWriter` is limited to copying, merging, cropping, and transforming pages from other PDFs into new ones.
- `append()` method adds pages to the end of the new pdf, where `merge()` has an additional integer as the first argument that specifies where to insert the pages. `writer.merge(2, 'Recursion_Chapter1.pdf', (0, 5))` copies pages 0-4 and inserts them at index 2 (after the third page). The original page at index 2 and all pages afterwards get shifted back.
- `Page` objects have a `rotate()` method that you pass either 90, 180, or 270 to rotate clockwise or -90, -180, or -270 to rotate counterclockwise.
- You can also append pages, add watermarks and overlays, encrypt and decrypt pdf's, and combine select pages from many pdfs into a single document. Review the chapter in the book or lookup `pypdf` documentation if you need to do more complex pdf tasks.

Word

- `python-docx` (installed via pip) handles Word documents and is imported via `import docx`
- You work with docx via a `Document` object that contians a list of `Paragraph` objects. Each `Paragraph` object contains a list of one or more `Run` objects. A `Run` object is a continuous run of text with the same style. Both `Paragraph` and `Run` objects have a `text` attribute that gives you their full text.

```
import docx
def get_text(filename):
    doc = docx.Document(filename)
    full_text = []
    for para in doc.paragraphs:
        full_text.append(para.text)
    return '\n'.join(full_text)
```

- It's also worth noting that `Paragraph` and `Run` objects have a `style` attribute that you can set to a number of style names which are string values such as 'Title', 'Subtitle', or 'Body Text'. The book has many more if you ever need to work with a word document. For paragraphs you use the string as is but for runs you append ' Char'. Runs have a variety of attributes for other styling such as `bold` and `underline`.
- See the book or search for `python-docx` documentation for more on Word documents.

Chapter 18 - CSV, JSON, and XML Files

- Python comes with `csv`, `json`, and `xml` modules in the stdlib.

CSV

- CSV files are simple and while you could parse them yourself, using the `csv` library handles edge cases gracefully
- To read a csv file, open it normally with `open` then use a `csv.reader()` on the open file handle, then iterate over the lines or force them to a `list` (which makes it a list of lists) to consume at once. Don't forget to `close()` your file handle.
- Two GOTCHAs: you can loop over a csv file only once and need to reopen to iterate by row again, and the `line_num` attribute of a csv reader starts at 1 not 0 even though when you index into a list of lists of csv data you index normally for lists starting at [0][0].

- `csv.writer()` works similar. You still call `open` but for `'w'` and it has a `writerow` method that you give a list of values to write. On Windows, you need to pass `newline=''` to `open` for technical reasons.
- `csv.writer()` has keyword arguments for `delimiter` (perhaps you want to change to '\t') and `lineterminator` ('\n\n' to double space your writer).
- If a csv has a header, you can use `csv.DictReader()` on the open file handle to load it as a list of dictionary elements for each row. Then you don't need to index anything if you iterate over row objects:

```
import csv
example_file = open('exampleWithHeader3.csv')
example_dict_reader = csv.DictReader(example_file)
for row in example_dict_reader:
    print(row['Timestamp'], row['Fruit'], row['Quantity'])
```

- There is also a `csv.DictWriter(output_file, header_list)` method that can take dictionaries in its `writerow(dict)` method. Remember to call `writerow()` for the first row if you want to save the header to the output.

JSON

- The main workhorse functions are `json.loads()` and `json.dumps()`
- json can't store every Python datatype, only: strings, integers, floats, Booleans, lists, dictionaries, and NoneType
- Full json Library Documentation

XML

- Python has `xml.dom`, `xml.sax`, and `xml.etree.ElementTree` modules for handling XML text. DOM and ElementTree modules read the entire file into memory at once where SAX reads the XML file as a stream of elements and is more suitable for larger files.

```
import xml.etree.ElementTree as ET
tree = ET.parse("my_data.xml")
root = tree.getroot()
for elem in root.iter():
    print(elem.tag, '--', elem.text)
```

- ElementTree Documentation
- There is also an `xmltodict` module to convert xml to a Python dictionary.
- `Element` objects have `tag` and `text` attributes

YAML (not in book)

```
import yaml  # pip install pyyaml

# Reading
with open("config.yaml", "r") as f:
    config = yaml.safe_load(f)

# Writing
data = {"host": "localhost", "port": 8080, "debug": False}
with open("config.yaml", "w") as f:
    yaml.dump(data, f)

# From/to string
config = yaml.safe_load("host: localhost\nport: 8080")
yaml_string = yaml.dump(data)
```

- pyyaml Documentation
- Always use `safe_load`

TOML (not in book)

```
import tomllib

with open("pyproject.toml", "rb") as f:
    config = tomllib.load(f)
```

- tomllib Documentation
- TOML Website has more information on the format.

Chapter 19 - Keeping Time, Scheduling Tasks, and Launching Programs

Date and Time

- `time.time()` returns the epoch timestamp, and `time.sleep(seconds)` which pauses a program.
- `time.ctime()` gives a human readable string of time, and `time.ctime(timestamp)` gives a human readable string of any Unix timestamp.
- For more general use of time, use the `datetime` library. `datetime.datetime.now()` returns a `datetime.datetime`. You can assign it to `dt` and access `dt.year`, `dt.month`, and `dt.day` along with `dt.hour`, `dt.minute`, and `dt.second`. You can create a `datetime.datetime` for any moment by passing it the year, month, day, hour, and second like `datetime.datetime(2026, 10, 21, 16, 29, 0)`. You can convert a epoch timestamp with `datetime.datetime.fromtimestamp(timestamp)`. Time will be in the local timezone. `datetime.datetime` objects can be compared with comparison operators and the greater value comes after the lesser.
- `datetime` also provides a `timedelta` method so you can calculate `delta = datetime.timedelta(days=11, hours=10, minutes=9, seconds=8)`. A timedelta has `days`, `seconds`, `microseconds` attributes and a `total_seconds()` method. `str(delta)` will return "11 days, 10:09:08". You can use arithmetic operators between `datetime` values and `timedelta` objects and between two `timedelta`. A `timedelta` does not include `months` and `years` arguments since these vary in duration.
- Use the `strftime()` method to display a `datetime` object as a string. For example `datetime.datetime.now().strftime('%Y/%m/%d %H:%M:%S')` will format now like YYYY/MM/DD HH:mm:ss. For more formatting options, see strftime.org. The f is for format.
- You can convert a string to a datetime with `datetime.datetime.strptime('October 21, 2026', '%B %d, %Y')`. The p is for Python.

Launching Programs

- `subprocess.run(['C:\\Windows\\System32\\calc.exe'])` will launch a calculator on Windows, blocking until Calc exits
- `subprocess.run(['/usr/bin/gnome-calculator'])` will do the same in Ubuntu
- `subprocess.run(['open', '/System/Applications/Calculator.app'])` will in Mac OS X
- `subprocess.Popen(cmd_list)` works like `run` but doesn't wait on the program to exit

- `subprocess.Popen(cmd_list)` returns a `Popen` object with `poll()` and `wait()` methods. `poll()` returns `None` if the process is still running and the exit code if it has terminated. `wait()` does likewise but blocks until the process has terminated rather than returning `None`. You can call the `kill()` method of the `Popen` object to kill the process.
- `run(cmd_list)` and `Popen(cmd_list)` take a list where the first value is the executable and any other values are command line arguments to the program you want to launch.
- `proc = subprocess.run(['ping', '-c', '4', 'nostarch.com'], capture_output=True, text=True)` will capture the output of `ping` in `proc.stdout`
- You can open files in their default application by launching `start` on Windows with the file as argument and `open` on MacOS or Linux with the file as argument. On windows you also need to pass `shell=True`
- To run a program at a specific time, you are best off using Task Scheduler on Windows, launchd on MacOS, or cron on Linux.

Chapter 20 - Sending Email, Texts, and Push Notifications

- To use EZGmail, you need to sign up for a Google Cloud account and enable the Gmail API.
- It's easiest to send sms like email through an sms email gateway.
- ntfy is a more reliable way to get push notifications on your phone than an sms gateway.

```
import requests, time
requests.post('https://ntfy.sh/YourTopicHere', 'Hello, world!')
time.sleep(1)
requests.post('https://ntfy.sh/YourTopicHere', 'The rent is too high!',
              headers={'Title':'Important: Read this!',
                       'Tags': 'warning,neutral_face', 'Priority':'5'})
```

- It is also possible to poll a ntfy topic for new messages with a Python app

Chapter 21 - Making Graphs and Manipulating Images

- `pillow` (available via pip) allows you to manipulate images.
- `matplotlib` creats a wide variety of graphs of professional quality.
- Full matplotlib Documentation
- In `matplotlib` the words *plot*, *graph*, and *chart* are used interchangeably, and the term *figure* refers to the window that contains one or more plots.

```
import matplotlib.pyplot as plt
x_values = [0, 1, 2, 3, 4, 5]
y_values1 = [10, 13, 15, 18, 16, 20]
y_values2 = [9, 11, 18, 16, 17, 19]
plt.plot(x_values, y_values1)
plt.plot(x_values, y_values2)
plt.savefig('linegraph.png')
plt.show()  # Opens a window with the plot
```

- To save an image, you must call `savefig(filename)` before `show()`, but from the show window there is an interactive option to save the graph as an image.
- Instead of a linegraph as above, you can do a scatterplot by calling `plt.scatter(x_values, y_values1)`.
- You can also do a bar graph with categories and values:

```
import matplotlib.pyplot as plt
categories = ['Cats', 'Dogs', 'Mice', 'Moose']
values = [100, 200, 300, 400]
plt.bar(categories, values)
plt.savefig('bargraph.png')
plt.show()
```

- If you close the show window it resets the graph data and you must plot it again before callings `show()` again.
- You can also graph a pie chart with `.pie(slices, labels=labels, autopct='%.1f%%')` where `slices` is the size of each slice, `labels` the name of each slice, and `autopct` a format specifier that tells how many places to show after the decimal point. If you leave out `autopct` it doesn't show percentage text. With `autopct=%.1f%%` a `slices` value of `100, 200, 300, 400` is interpreted as 10.0%, 20.0%, 30.0%, and 40.0%.
- A more feature rich line graph is displayed as follows:

```
import matplotlib.pyplot as plt
x_value = [0, 1, 2, 3, 4, 5]
y_values1 = [10, 13, 15, 18, 16, 20]
y_values2 = [9, 11, 18, 16, 17, 19]
plt.plot(x_values, y_values1, marker='o', color='b', label='Line 1')
plt.plot(x_values, y_values1, marker='s', color='r', label='Line 2')
plt.legend()
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Graph Title')
plt.grid(True)
plt.show()
```

- `marker` creates a point for each value on the line. 'o' specifies a O-shaped circle and 's' specifies a square. For `color`, 'b' is blue and 'r' is red.
- `matplotlib` is capable of much more graphing, included 3d graphs.

Chapter 22 - Recognizing Text in Images

- I read the remaining three chapters but didn't find anything noteworthy unless I end up with an OCR project or want to do text-to-speech for fun

Chapter 23 - Controlling the Keyboard and Mouse

Chapter 24 - Text-to-Speech and Speech Recognition Engines

The End

And that's it for this book. It was a pretty cursory introduction to Python (less depth than the official Python Tutorial) followed by a lot of interesting use cases for various packages from pip. You can probably tell from my notes that I found Platwright, SQLite, JSON and other text formats, and Python date and time functions the most practical and had some interest in matplotlib.